

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA · INFORMÁTICA Y SISTEMAS

PROYECTO DE PROGRAMACIÓN AVANZADA

Inserción y Quicksort

Comparación experimental según el tamaño y el patrón de entrada

5 patrones
2 algoritmos

Natalia Lizeth Huanca Quispe

PREGUNTA DEL PROYECTO

La estructura de los datos cambia el costo

¿Cómo varían el tiempo y las operaciones al cambiar el tamaño y el orden inicial de la entrada?

INSERCIÓN

Avanza por la lista y mueve cada elemento hasta su posición. Si los datos ya están ordenados, hace una sola pasada.

Mejor caso $O(n)$ · Peor caso $O(n^2)$

QUICKSORT

Elige un pivote, particiona la lista y ordena recursivamente las dos partes. La mediana de tres ayuda a elegir el pivote.

Promedio $O(n \log n)$ · Peor caso $O(n^2)$

No hay un ganador universal: la entrada ascendente favorece a inserción; las entradas grandes y desordenadas favorecen a Quicksort.

RECORRIDO DEL CÓDIGO

Qué hace cada implementación

Dos rutinas distintas; una misma entrada para comparar su comportamiento.

src/insercion.cpp

```
for (j = i; j > 0; --j) {
  if (a[j-1] <= a[j]) break;
  swap(a[j-1], a[j]);
}
```

Intercambia vecinos fuera de orden.
El contador suma comparaciones e intercambios.

src/quicksort.cpp

```
pivot = mediana(a[low], a[mid], a[high]);
cut = particion_Hoare(a, pivot);
quicksort(low, cut);
quicksort(cut + 1, high);
```

Cuenta comparaciones, intercambios y profundidad máxima de recursión.

Inserción: estable

Quicksort: inestable

Mediana de tres mejora la selección del pivote; no elimina $O(n^2)$ en el peor caso.

CÓMO SE EJECUTA

main.cpp controla una medición reproducible

La generación, el cronómetro y los contadores están separados para que la comparación sea justa.

01 Configurar
semilla, n, repeticiones

02 Generar
patrón y semilla efectiva

03 Copiar
misma base para ambos algoritmos

04 Cronometrar
solo ordenar con steady_clock

05 Validar
confirmar que quedó ordenado

06 Contar
segunda corrida con métricas

07 Guardar
una fila CSV por algoritmo

RELOJ

No mide generación, contadores ni escritura CSV.

SEMILLA

Derivada por tamaño, patrón y repetición; se registra en cada fila.

```
CSV: algoritmo · n · patrón · repetición · tiempo · comparaciones · intercambios · profundidad · semilla · commit
```

CÓMO SE MIDIERON

Diseño experimental y equipo

Los dos algoritmos reciben copias de las mismas entradas generadas.

5 PATRONES DE ENTRADA

Aleatorio	Enteros uniformes entre 0 y 1 000 000
Ascendente / descendente	Versiones ordenadas de una muestra aleatoria
Casi ordenado	Se permuta el 5 % de las posiciones
Duplicados	15 valores posibles

EQUIPO DE MEDICIÓN

AMD Ryzen 5 7530U
6 núcleos · 12 hilos · 2.0 GHz base

16 GB RAM · Windows 10.0.26200
g++ 14.2.0 (MSYS2 UCRT64)
C++17 · optimización -O2

2 tamaños × 5 patrones × 30 repeticiones × 2 algoritmos = 600 mediciones

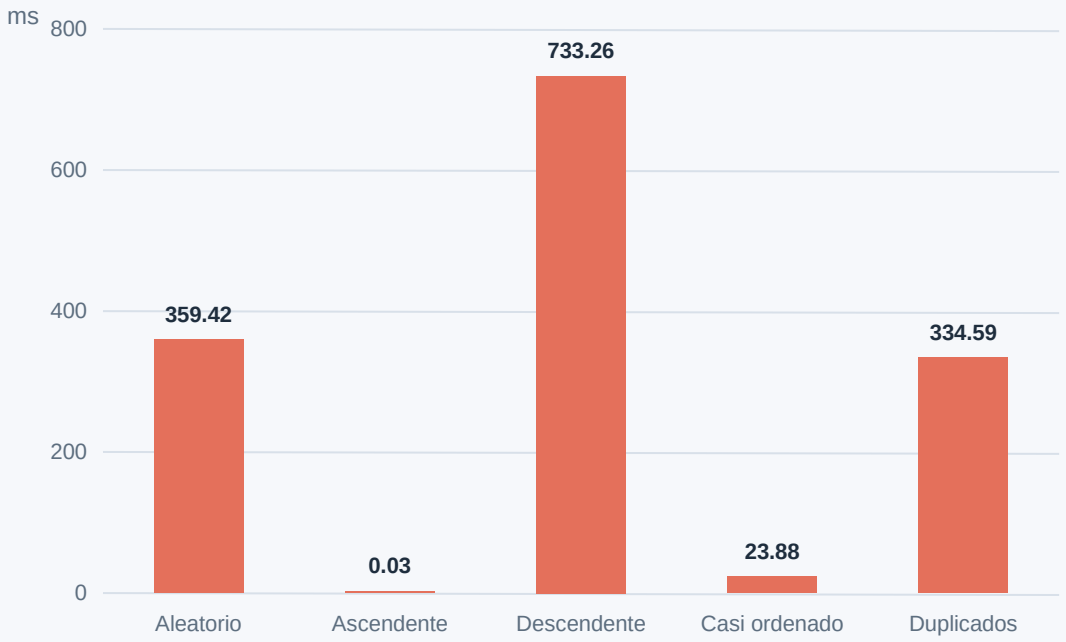
n = 1 000 y 10 000 · semilla base 12345
Inserción se limitó a n = 10 000 por su costo cuadrático.

RESULTADOS

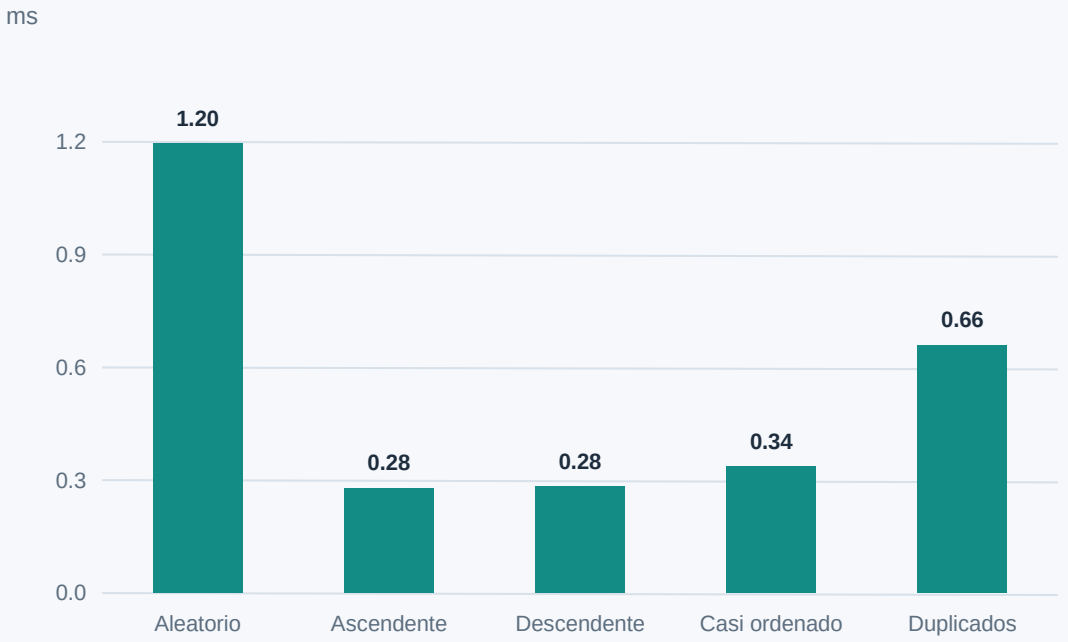
A n = 10 000, los perfiles se separan

Tiempo medio de 30 corridas en milisegundos · cada gráfico usa su propia escala.

Inserción



Quicksort



Inserción: 0,032 ms en ascendente y 733,264 ms en descendente. Quicksort: entre 0,279 y 1,196 ms.

INTERPRETACIÓN

La ventaja depende del patrón

Razón = media de inserción / media de Quicksort. Valores mayores que 1 favorecen a Quicksort.

Patrón	Inserción (ms)	Quicksort (ms)	Razón	Gana
Aleatorio	359.419 ± 6.691	1.196 ± 0.070	**300.5×**	**Quicksort**
Ascendente	0.032 ± 0.007	0.279 ± 0.058	**0.1×**	**Inserción**
Descendente	733.264 ± 52.649	0.284 ± 0.060	**2584.7×**	**Quicksort**
Casi ordenado	23.879 ± 1.134	0.336 ± 0.078	**71.0×**	**Quicksort**
Duplicados	334.595 ± 3.672	0.660 ± 0.087	**507.2×**	**Quicksort**

MÉTRICAS DE OPERACIÓN · DESCENDENTE · n = 10 000

Inserción 49,995 M comparaciones · 49,995 M intercambios

Quicksort 163,666 comparaciones · 5 050 intercambios · profundidad 14

Quicksort ganó en 4 de 5 patrones; inserción fue ~8,6× más rápida en orden ascendente.